



طراحی الگوریتم‌ها – جریان بیشینه

Introduction to Algorithm

مهدی جوانمردی

فصل ۲۶ کتاب – جریان بیشینه

• ۲۶.۱: شبکه جریان

• ۲۶.۲: الگوریتم Ford-Fulkerson

26 Maximum Flow 708

26.1 Flow networks 709

26.2 The Ford-Fulkerson method 714

26.3 Maximum bipartite matching 732

شبکه‌های جریان Flow Networks

Use a graph to model material that flows through conduits.

Each edge represents one conduit, and has a **capacity**, which is an upper bound on the **flow rate** = units/time.

Can think of edges as pipes of different sizes. But flows don't have to be of liquids. Book has an example where a flow is how many trucks per day can ship hockey pucks between cities.

Want to compute max rate that we can ship material from a designated **source** to a designated **sink**.

Flow networks

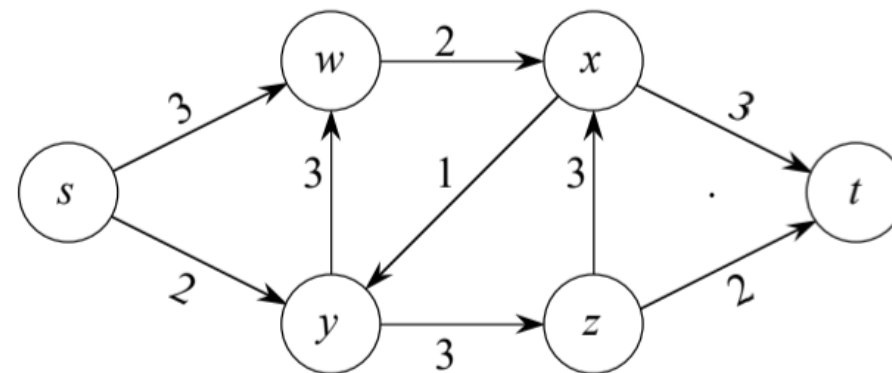
$G = (V, E)$ directed.

Each edge (u, v) has a **capacity** $c(u, v) \geq 0$.

If $(u, v) \notin E$, then $c(u, v) = 0$.

If $(u, v) \in E$, then reverse edge $(v, u) \notin E$. (Can work around this restriction.)

Source vertex s , **sink** vertex t , assume $s \rightsquigarrow v \rightsquigarrow t$ for all $v \in V$, so that each vertex lies on a path from source to sink.



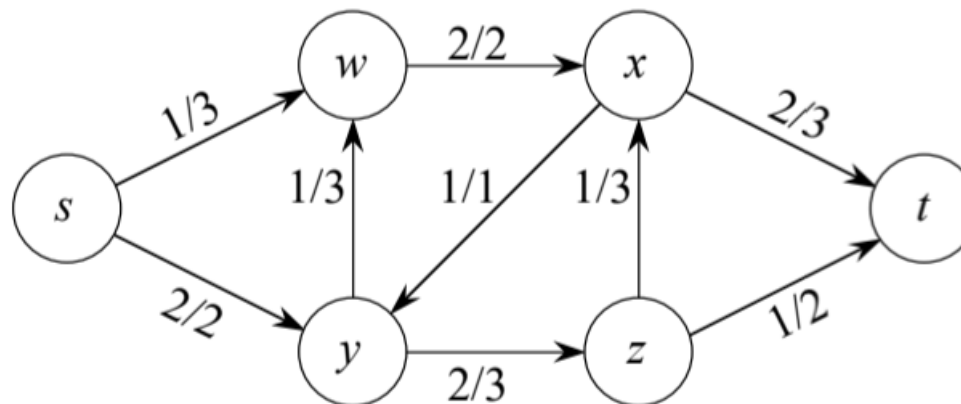
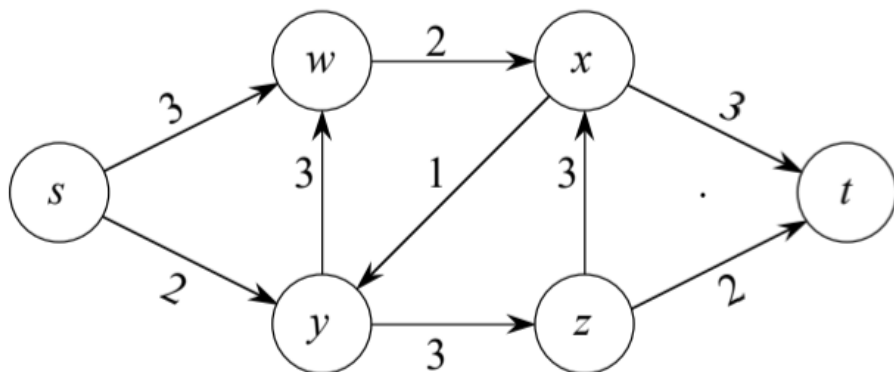
شبکه‌های جریان Flow Networks

Flow

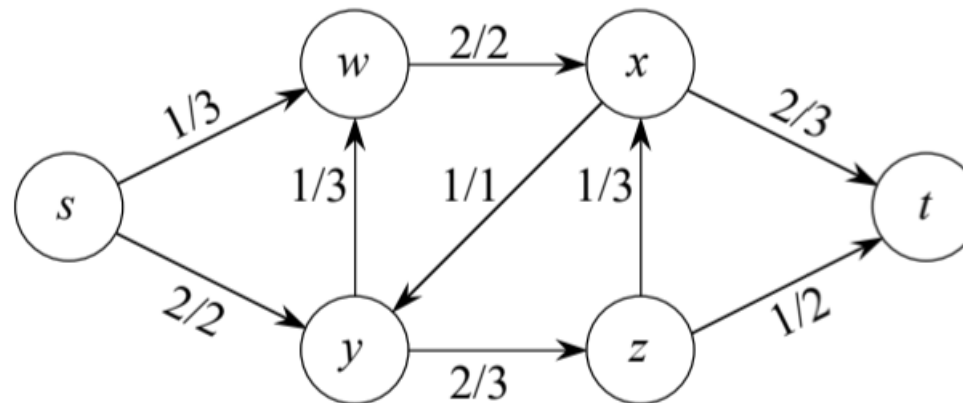
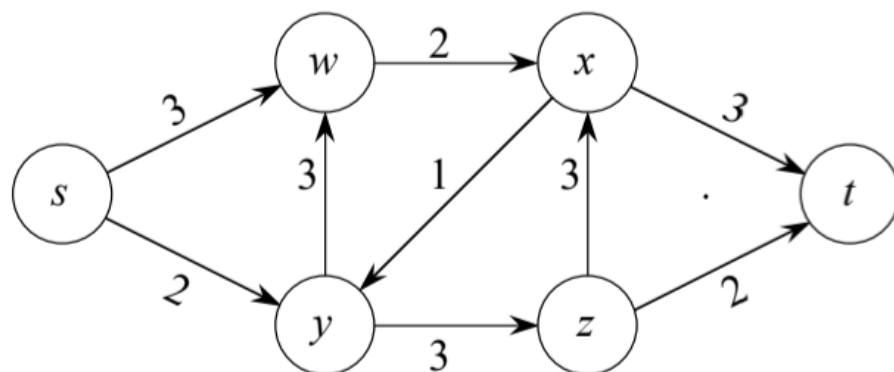
A function $f : V \times V \rightarrow \mathbb{R}$ satisfying

- **Capacity constraint:** For all $u, v \in V$, $0 \leq f(u, v) \leq c(u, v)$,
- **Flow conservation:** For all $u \in V - \{s, t\}$, $\underbrace{\sum_{v \in V} f(v, u)}_{\text{flow into } u} = \underbrace{\sum_{v \in V} f(u, v)}_{\text{flow out of } u}$.

Equivalently, $\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) = 0$.



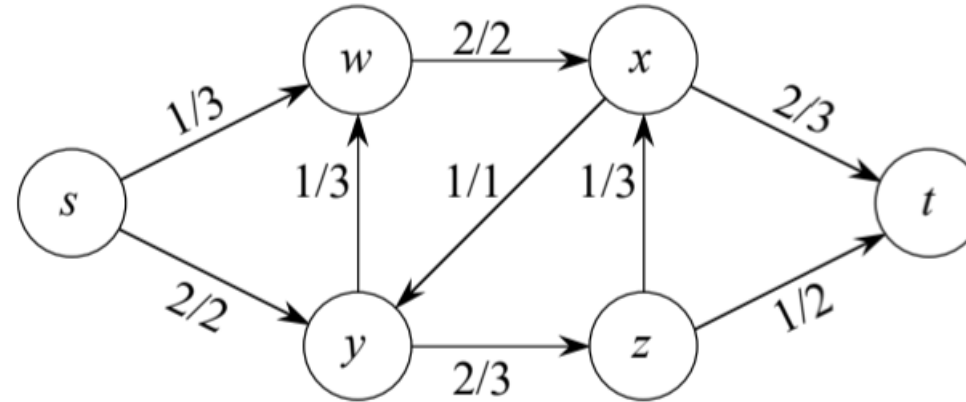
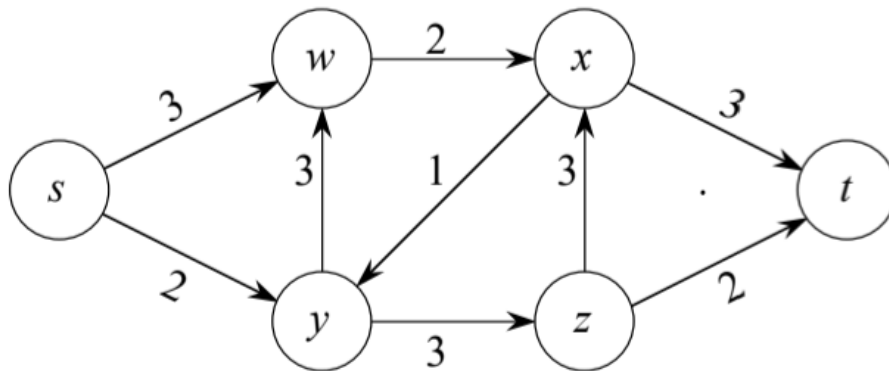
شبکه‌های جریان Flow Networks



$$\begin{aligned}
 \text{Value of flow } f &= |f| \\
 &= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) \\
 &= \text{flow out of source} - \text{flow into source} .
 \end{aligned}$$

In the example above, value of flow $f = |f| = 3$.

مسئله جریان بیشینه



Maximum-flow problem

Given G , s , t , and c , find a flow whose value is maximum.

Antiparallel edges

Definition of flow network does not allow both (u, v) and (v, u) to be edges. These edges would be *antiparallel*.

What if we really need antiparallel edges?

مسئله جریان بیشینه

Maximum-flow problem

Given G , s , t , and c , find a flow whose value is maximum.

Antiparallel edges

Definition of flow network does not allow both (u, v) and (v, u) to be edges. These edges would be *antiparallel*.

What if we really need antiparallel edges?

- Choose one of them, say (u, v) .
- Create a new vertex v' .
- Replace (u, v) by two new edges (u, v') and (v', v) , with $c(u, v') = c(v', v) = c(u, v)$.
- Get an equivalent flow network with no antiparallel edges.

برش در شبکه جریان

A **cut** (S, T) of flow network $G = (V, E)$ is a partition of V into S and $T = V - S$ such that $s \in S$ and $t \in T$.

- Similar to cut used in minimum spanning trees, except that here the graph is directed, and we require $s \in S$ and $t \in T$.

For flow f , the **net flow** across cut (S, T) is

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) .$$

Capacity of cut (S, T) is

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v) .$$

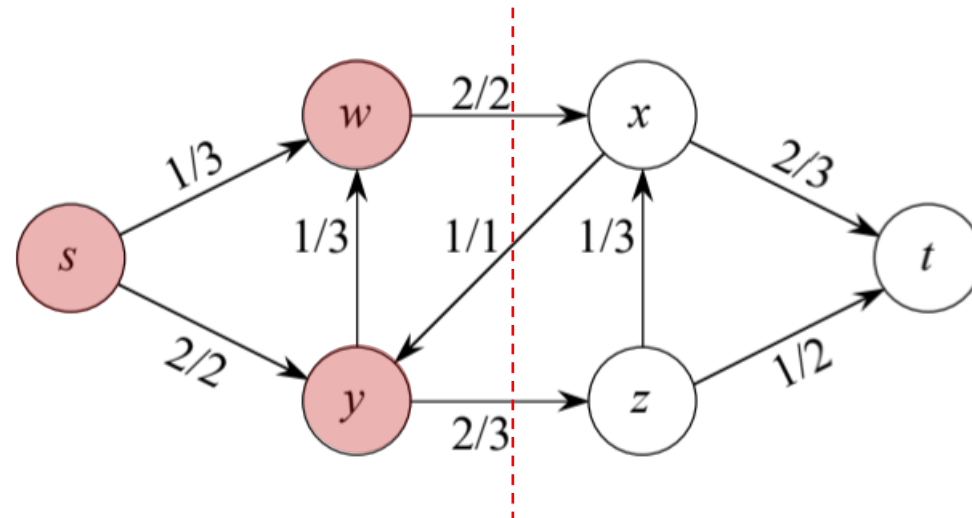
A **minimum cut** of G is a cut whose capacity is minimum over all cuts of G .

مثال برش در شبکه جریان

In previous example, consider the cut $S = \{s, w, y\}$, $T = \{x, z, t\}$.

$$\begin{aligned} f(S, T) &= \underbrace{f(w, x) + f(y, z)}_{\text{from } S \text{ to } T} - \underbrace{f(x, y)}_{\text{from } T \text{ to } S} \\ &= 2 + 2 - 1 \\ &= 3. \end{aligned}$$

$$\begin{aligned} c(S, T) &= \underbrace{c(w, x) + c(y, z)}_{\text{from } S \text{ to } T} \\ &= 2 + 3 \\ &= 5. \end{aligned}$$



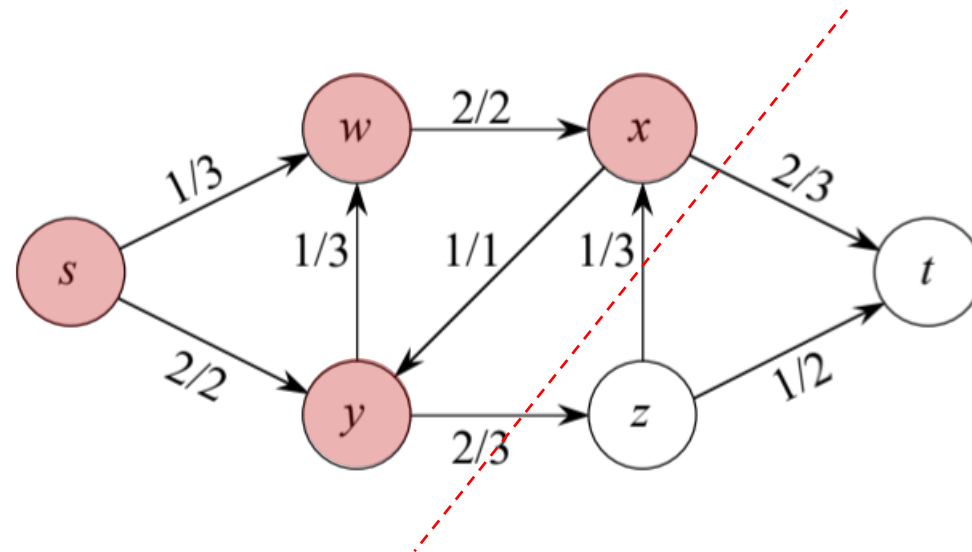
مثال برش در شبکه جریان

Now consider the cut $S = \{s, w, x, y\}, T = \{z, t\}$.

$$\begin{aligned} f(S, T) &= \underbrace{f(x, t) + f(y, z)}_{\text{from } S \text{ to } T} - \underbrace{f(z, x)}_{\text{from } T \text{ to } S} \\ &= 2 + 2 - 1 \\ &= 3. \end{aligned}$$

$$\begin{aligned} c(S, T) &= \underbrace{c(x, t) + c(y, z)}_{\text{from } S \text{ to } T} \\ &= 3 + 3 \\ &= 6. \end{aligned}$$

Same flow as previous cut, higher capacity.



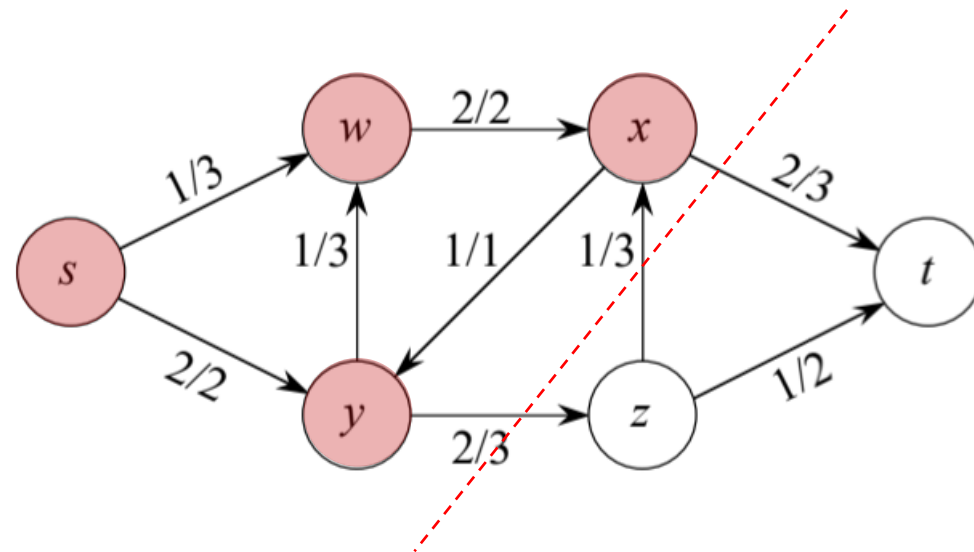
لم برش در شبکه جریان

Lemma

For any cut (S, T) , $f(S, T) = |f|$.
(Net flow across the cut equals value of the flow.)

Corollary

The value of any flow $\leq$ capacity of any cut.



اثبات لم برش در شبکه جریان

Proof Rewrite flow conservation: for any $u \in V - \{s, t\}$,

$$\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) = 0.$$

Take definition of $|f|$ and add in left-hand side of above equation, summed over all vertices in $S - \{s\}$. Above equation applies to each vertex in $S - \{s\}$ (since $t \notin S$ and obviously $s \notin S - \{s\}$), so just adding in lots of 0s:

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S - \{s\}} \left(\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) \right).$$

Expand right-hand summation and regroup terms:

$$\begin{aligned} |f| &= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S - \{s\}} \sum_{v \in V} f(u, v) - \sum_{u \in S - \{s\}} \sum_{v \in V} f(v, u) \\ &= \sum_{v \in V} \left(f(s, v) + \sum_{u \in S - \{s\}} f(u, v) \right) - \sum_{v \in V} \left(f(v, s) + \sum_{u \in S - \{s\}} f(v, u) \right) \\ &= \sum_{v \in V} \sum_{u \in S} f(u, v) - \sum_{v \in V} \sum_{u \in S} f(v, u). \end{aligned}$$

Lemma

For any cut (S, T) , $f(S, T) = |f|$.

(Net flow across the cut equals value of the flow.)

اثبات لم برش در شبکه جریان

Lemma

For any cut (S, T) , $f(S, T) = |f|$.
(Net flow across the cut equals value of the flow.)

Partition V into $S \cup T$ and split each summation over V into summations over S and T :

$$\begin{aligned} |f| &= \sum_{v \in S} \sum_{u \in S} f(u, v) + \sum_{v \in T} \sum_{u \in S} f(u, v) - \sum_{v \in S} \sum_{u \in S} f(v, u) - \sum_{v \in T} \sum_{u \in S} f(v, u) \\ &= \sum_{v \in T} \sum_{u \in S} f(u, v) - \sum_{v \in T} \sum_{u \in S} f(v, u) \\ &\quad + \left(\sum_{v \in S} \sum_{u \in S} f(u, v) - \sum_{v \in S} \sum_{u \in S} f(v, u) \right). \end{aligned}$$

Summations within parentheses are the same, since $f(x, y)$ appears once in each summation, for any $x, y \in V$. These summations cancel:

$$\begin{aligned} |f| &= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) \\ &= f(S, T). \end{aligned}$$

Corollary

The value of any flow $\leq$ capacity of any cut.

اثبات نتیجه لم برش در شبکه جریان

Lemma

For any cut (S, T) , $f(S, T) = |f|$.
(Net flow across the cut equals value of the flow.)

Corollary

The value of any flow $\leq$ capacity of any cut.

Proof Let (S, T) be any cut, f be any flow.

$$\begin{aligned}
 |f| &= f(S, T) && \text{(lemma)} \\
 &= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) && \text{(definition of } f(S, T)) \\
 &\leq \sum_{u \in S} \sum_{v \in T} f(u, v) && (f(v, u) \geq 0) \\
 &\leq \sum_{u \in S} \sum_{v \in T} c(u, v) && \text{(capacity constraint)} \\
 &= c(S, T). && \text{(definition of } c(S, T)) \quad \blacksquare \text{ (corollary)}
 \end{aligned}$$

Therefore, maximum flow $\leq$ capacity of minimum cut.

Will see a little later that this is in fact an equality.

الگوریتم فورد-فولکرسون Ford-Fulkerson

Residual network

Given a flow f in network $G = (V, E)$.

Consider a pair of vertices $u, v \in V$.

How much additional flow can we push directly from u to v ?

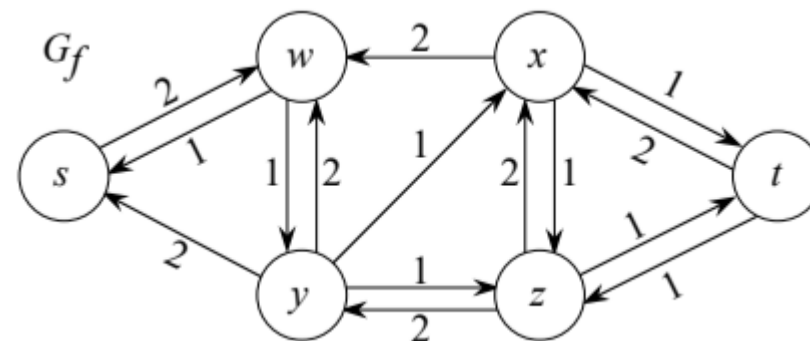
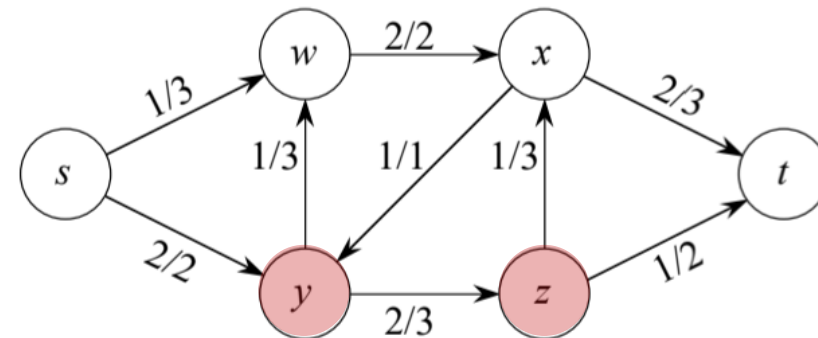
That's the **residual capacity**,

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{if } (u, v) \in E, \\ f(v, u) & \text{if } (v, u) \in E, \\ 0 & \text{otherwise (i.e., } (u, v), (v, u) \notin E \text{).} \end{cases}$$

The **residual network** is $G_f = (V, E_f)$, where

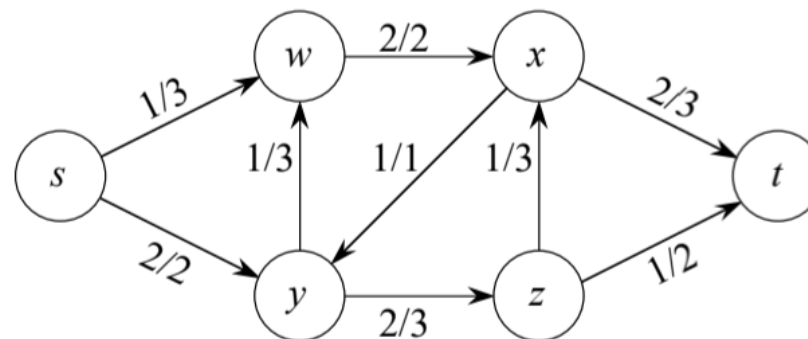
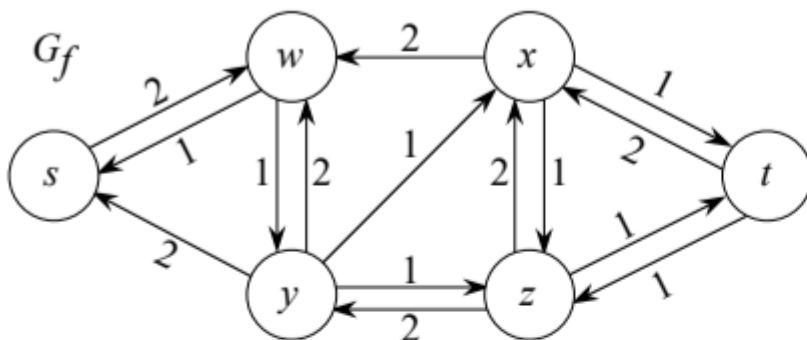
$$E_f = \{(u, v) \in V \times V : c_f(u, v) > 0\}.$$

Each edge of the residual network can admit a positive flow.



الگوریتم فورد-فولکرسون Ford-Fulkerson

For our example:



Every edge $(u, v) \in E_f$ corresponds to an edge $(u, v) \in E$ or $(v, u) \in E$ (or both).

Therefore, $|E_f| \leq 2|E|$.

Residual network is similar to a flow network, except that it may contain antiparallel edges $((u, v)$ and $(v, u))$. Can define a flow in a residual network that satisfies the definition of a flow, but with respect to capacities c_f in G_f .

Given flows f in G and f' in G_f , define $(f \uparrow f')$, the **augmentation** of f by f' , as a function $V \times V \rightarrow \mathbb{R}$:

$$(f \uparrow f')(u, v) = \begin{cases} f(u, v) + f'(u, v) - f'(v, u) & \text{if } (u, v) \in E, \\ 0 & \text{otherwise} \end{cases}$$

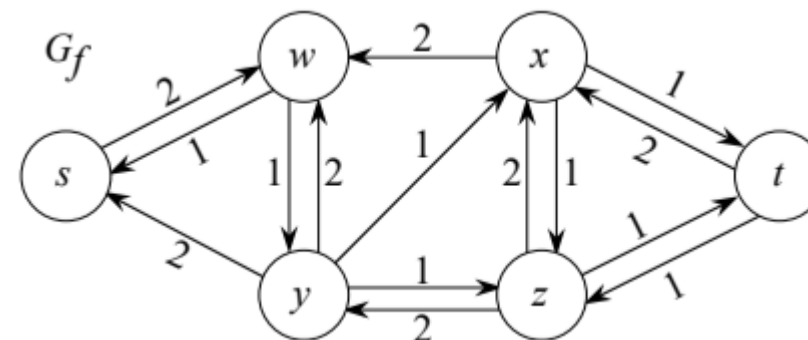
for all $u, v \in V$.

الگوریتم فورد-فولکرسون Ford-Fulkerson

$$(f \uparrow f')(u, v) = \begin{cases} f(u, v) + f'(u, v) - f'(v, u) & \text{if } (u, v) \in E, \\ 0 & \text{otherwise} \end{cases}$$

for all $u, v \in V$.

Intuition: Increase the flow on (u, v) by $f'(u, v)$ but decrease it by $f'(v, u)$ because pushing flow on the reverse edge in the residual network decreases the flow in the original network. Also known as **cancellation**.



Lemma 26.1

Let $G = (V, E)$ be a flow network with source s and sink t , and let f be a flow in G . Let G_f be the residual network of G induced by f , and let f' be a flow in G_f . Then the function $f \uparrow f'$ defined in equation (26.4) is a flow in G with value $|f \uparrow f'| = |f| + |f'|$.

الگوریتم فورد-فولکرسون Ford-Fulkerson

Proof We first verify that $f \uparrow f'$ obeys the capacity constraint for each edge in E and flow conservation at each vertex in $V - \{s, t\}$.

For the capacity constraint, first observe that if $(u, v) \in E$, then $c_f(v, u) = f(u, v)$. Therefore, we have $f'(v, u) \leq c_f(v, u) = f(u, v)$, and hence

$$\begin{aligned} (f \uparrow f')(u, v) &= f(u, v) + f'(u, v) - f'(v, u) \quad (\text{by equation (26.4)}) \\ &\geq f(u, v) + f'(u, v) - f(u, v) \quad (\text{because } f'(v, u) \leq f(u, v)) \\ &= f'(u, v) \\ &\geq 0. \end{aligned}$$

In addition,

$$\begin{aligned} (f \uparrow f')(u, v) &= f(u, v) + f'(u, v) - f'(v, u) \quad (\text{by equation (26.4)}) \\ &\leq f(u, v) + f'(u, v) \quad (\text{because flows are nonnegative}) \\ &\leq f(u, v) + c_f(u, v) \quad (\text{capacity constraint}) \\ &= f(u, v) + c(u, v) - f(u, v) \quad (\text{definition of } c_f) \\ &= c(u, v). \end{aligned}$$

الگوریتم فورد-فولکرسون Ford-Fulkerson

Proof We first verify that $f \uparrow f'$ obeys the capacity constraint for each edge in E and flow conservation at each vertex in $V - \{s, t\}$.

For flow conservation, because both f and f' obey flow conservation, we have that for all $u \in V - \{s, t\}$,

$$\begin{aligned}\sum_{v \in V} (f \uparrow f')(u, v) &= \sum_{v \in V} (f(u, v) + f'(u, v) - f'(v, u)) \\ &= \sum_{v \in V} f(u, v) + \sum_{v \in V} f'(u, v) - \sum_{v \in V} f'(v, u) \\ &= \sum_{v \in V} f(v, u) + \sum_{v \in V} f'(v, u) - \sum_{v \in V} f'(u, v) \\ &= \sum_{v \in V} (f(v, u) + f'(v, u) - f'(u, v)) \\ &= \sum_{v \in V} (f \uparrow f')(v, u),\end{aligned}$$

where the third line follows from the second by flow conservation.

الگوریتم فورد-فولکرسون Ford-Fulkerson

Lemma 26.1

Let $G = (V, E)$ be a flow network with source s and sink t , and let f be a flow in G . Let G_f be the residual network of G induced by f , and let f' be a flow in G_f . Then the function $f \uparrow f'$ defined in equation (26.4) is a flow in G with value $|f \uparrow f'| = |f| + |f'|$.

Finally, we compute the value of $f \uparrow f'$. Recall that we disallow antiparallel edges in G (but not in G_f), and hence for each vertex $v \in V$, we know that there can be an edge (s, v) or (v, s) , but never both. We define $V_1 = \{v : (s, v) \in E\}$ to be the set of vertices with edges from s , and $V_2 = \{v : (v, s) \in E\}$ to be the set of vertices with edges to s . We have $V_1 \cup V_2 \subseteq V$ and, because we disallow antiparallel edges, $V_1 \cap V_2 = \emptyset$. We now compute

$$\begin{aligned} |f \uparrow f'| &= \sum_{v \in V} (f \uparrow f')(s, v) - \sum_{v \in V} (f \uparrow f')(v, s) \\ &= \sum_{v \in V_1} (f \uparrow f')(s, v) - \sum_{v \in V_2} (f \uparrow f')(v, s), \end{aligned} \quad (26.5)$$

الگوریتم فورد-فولکرسون Ford-Fulkerson

where the second line follows because $(f \uparrow f')(w, x)$ is 0 if $(w, x) \notin E$. We now apply the definition of $f \uparrow f'$ to equation (26.5), and then reorder and group terms to obtain

$$\begin{aligned}
 |f \uparrow f'| &= \sum_{v \in V_1} (f(s, v) + f'(s, v) - f'(v, s)) - \sum_{v \in V_2} (f(v, s) + f'(v, s) - f'(s, v)) \\
 &= \sum_{v \in V_1} f(s, v) + \sum_{v \in V_1} f'(s, v) - \sum_{v \in V_1} f'(v, s) \\
 &\quad - \sum_{v \in V_2} f(v, s) - \sum_{v \in V_2} f'(v, s) + \sum_{v \in V_2} f'(s, v) \\
 &= \sum_{v \in V_1} f(s, v) - \sum_{v \in V_2} f(v, s) \\
 &\quad + \sum_{v \in V_1} f'(s, v) + \sum_{v \in V_2} f'(s, v) - \sum_{v \in V_1} f'(v, s) - \sum_{v \in V_2} f'(v, s) \\
 &= \sum_{v \in V_1} f(s, v) - \sum_{v \in V_2} f(v, s) + \sum_{v \in V_1 \cup V_2} f'(s, v) - \sum_{v \in V_1 \cup V_2} f'(v, s). \quad (26.6)
 \end{aligned}$$

الگوریتم فورد-فولکرسون Ford-Fulkerson

$$= \sum_{v \in V_1} f(s, v) - \sum_{v \in V_2} f(v, s) + \sum_{v \in V_1 \cup V_2} f'(s, v) - \sum_{v \in V_1 \cup V_2} f'(v, s) . \quad (26.6)$$

In equation (26.6), we can extend all four summations to sum over V , since each additional term has value 0. (Exercise 26.2-1 asks you to prove this formally.) We thus have

$$\begin{aligned} |f \uparrow f'| &= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{v \in V} f'(s, v) - \sum_{v \in V} f'(v, s) \\ &= |f| + |f'| . \end{aligned} \quad (26.7) \quad \blacksquare$$

الگوریتم فورد-فولکرسون Ford-Fulkerson

Augmenting path

A simple path $s \rightsquigarrow t$ in G_f .

- Admits more flow along each edge.
- Like a sequence of pipes through which we can squirt more flow from s to t .

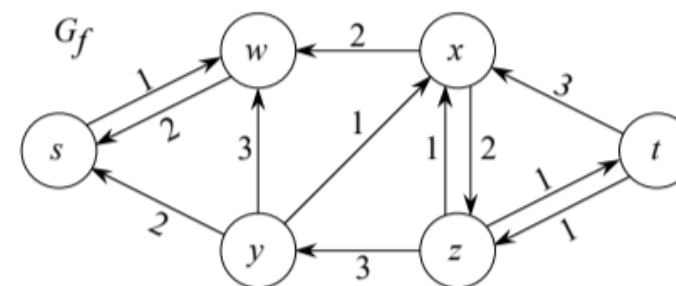
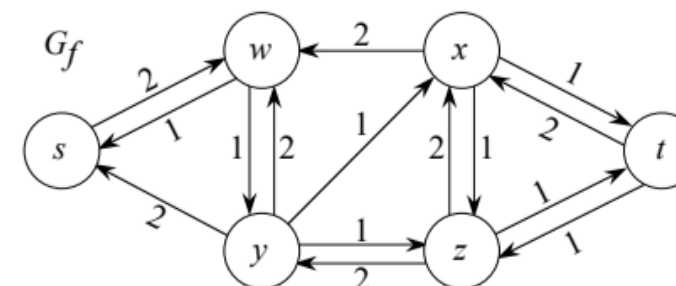
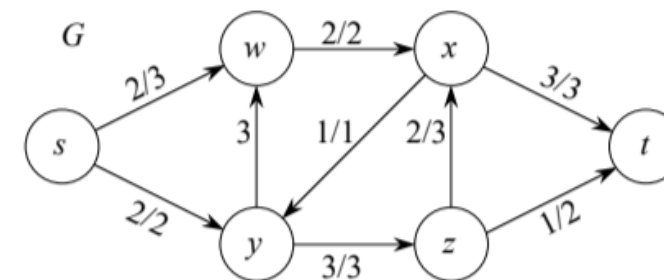
How much more flow can we push from s to t along augmenting path p ?

$$c_f(p) = \min \{c_f(u, v) : (u, v) \text{ is on } p\}.$$

For our example, consider the augmenting path $p = \langle s, w, y, z, x, t \rangle$.

Minimum residual capacity is 1.

After we push 1 additional unit along p : [Continue from G left on board from before. Edge (y, w) has $f(y, w) = 0$, which we omit, showing only $c(y, w) = 3$.]



Observe that G_f now has no augmenting path. Why? No edges cross the cut $(\{s, w\}, \{x, y, z, t\})$ in the forward direction in G_f . So no path can get from s to t .

Claim that the flow shown in G is a maximum flow.

الگوریتم فورد-فولکرسون Ford-Fulkerson

Lemma

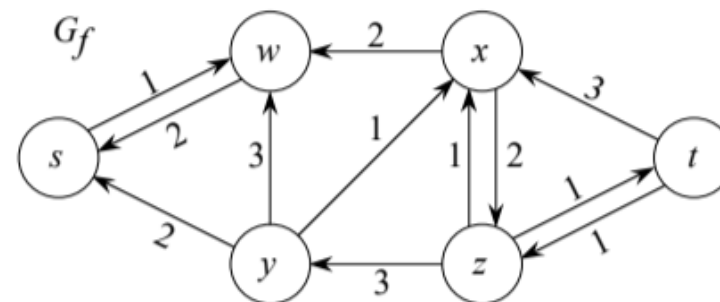
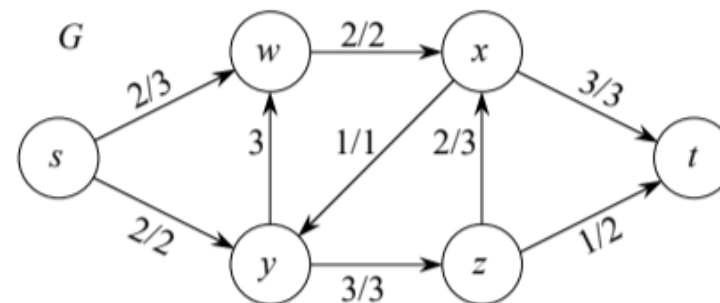
Given flow network G , flow f in G , residual network G_f . Let p be an augmenting path in G_f . Define $f_p : V \times V \rightarrow \mathbb{R}$:

$$f_p(u, v) = \begin{cases} c_f(p) & \text{if } (u, v) \text{ is on } p, \\ 0 & \text{otherwise.} \end{cases}$$

Then f_p is a flow in G_f with value $|f_p| = c_f(p) > 0$.

Corollary

Given flow network G , flow f in G , and an augmenting path p in G_f , define f_p as in lemma. Then $f \uparrow f_p$ is a flow in G with value $|f \uparrow f_p| = |f| + |f_p| > |f|$.



Theorem (Max-flow min-cut theorem)

The following are equivalent:

1. f is a maximum flow.
2. G_f has no augmenting path.
3. $|f| = c(S, T)$ for some cut (S, T) .

الگوریتم فورد-فولکرسون Ford-Fulkerson

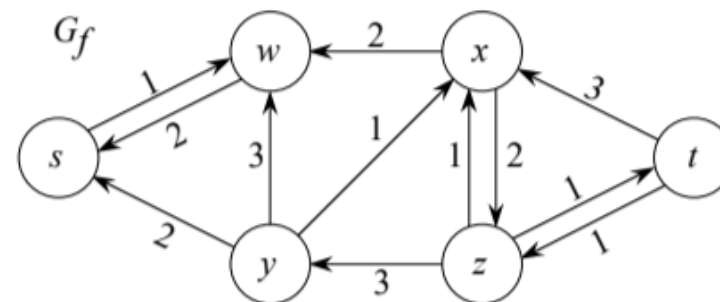
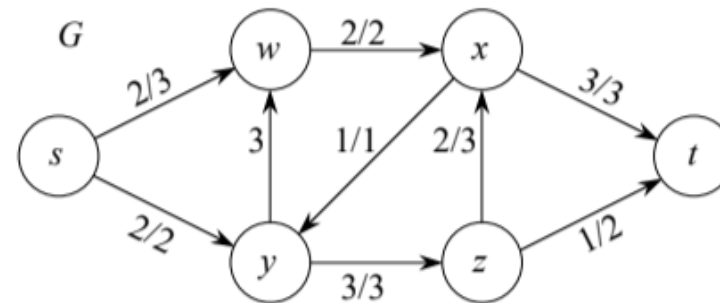
Theorem (Max-flow min-cut theorem)

The following are equivalent:

1. f is a maximum flow.
2. G_f has no augmenting path.
3. $|f| = c(S, T)$ for some cut (S, T) .

Proof

(1) $\Rightarrow$ (2): Show the contrapositive: if G_f has an augmenting path, then f is not a maximum flow. If G_f has augmenting path p , then by the above corollary, $f \uparrow f_p$ is a flow in G with value $|f| + |f_p| > |f|$, so that f was not a maximum flow.



الگوریتم فورد-فولکرسون Ford-Fulkerson

(2) $\Rightarrow$ (3): Suppose G_f has no augmenting path. Define

$S = \{v \in V : \text{there exists a path } s \rightsquigarrow v \text{ in } G_f\}$,

$T = V - S$.

Must have $t \in T$; otherwise there is an augmenting path.

Therefore, (S, T) is a cut.

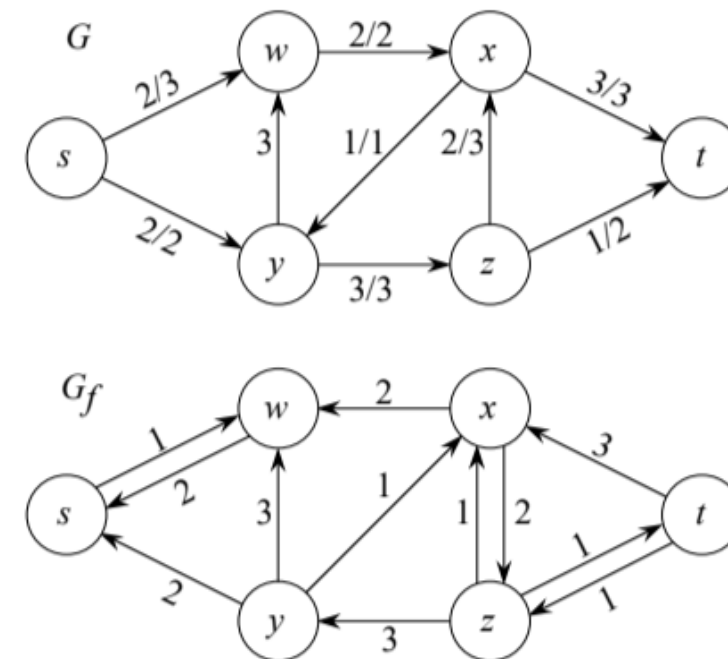
Consider $u \in S$ and $v \in T$:

- If $(u, v) \in E$, must have $f(u, v) = c(u, v)$; otherwise, $(u, v) \in E_f \Rightarrow v \in S$.
- If $(v, u) \in E$, must have $f(v, u) = 0$; otherwise, $c_f(u, v) = f(v, u) > 0 \Rightarrow (u, v) \in E_f \Rightarrow v \in S$.
- If $(u, v), (v, u) \notin E$, must have $f(u, v) = f(v, u) = 0$.

Then,

$$\begin{aligned} f(S, T) &= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{v \in T} \sum_{u \in S} f(v, u) \\ &= \sum_{u \in S} \sum_{v \in T} c(u, v) - \sum_{v \in T} \sum_{u \in S} 0 \\ &= c(S, T). \end{aligned}$$

By lemma, $|f| = f(S, T) = c(S, T)$.



Theorem (Max-flow min-cut theorem)

The following are equivalent:

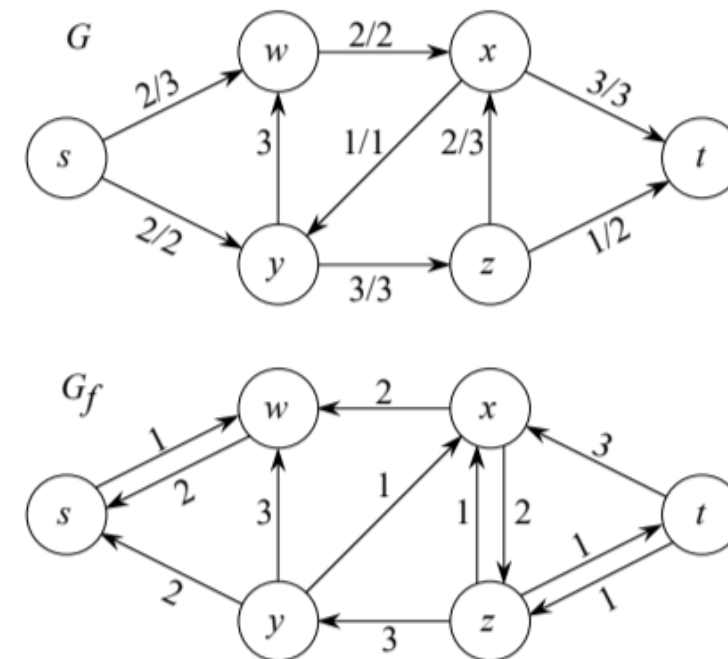
1. f is a maximum flow.
2. G_f has no augmenting path.
3. $|f| = c(S, T)$ for some cut (S, T) .

Lemma 26.4

Let f be a flow in a flow network G with source s and sink t , and let (S, T) be any cut of G . Then the net flow across (S, T) is $f(S, T) = |f|$.

الگوریتم فورد-فولکرسون Ford-Fulkerson

(3) $\Rightarrow$ (1): By corollary, $|f| \leq c(S, T)$.
Therefore, $|f| = c(S, T) \Rightarrow f$ is a max flow.



Lemma

For any cut (S, T) , $f(S, T) = |f|$.
(Net flow across the cut equals value of the flow.)

Corollary

The value of any flow $\leq$ capacity of any cut.

Theorem (Max-flow min-cut theorem)

The following are equivalent:

1. f is a maximum flow.
2. G_f has no augmenting path.
3. $|f| = c(S, T)$ for some cut (S, T) .

الگوریتم فورد-فولکرسون Ford-Fulkerson

Ford-Fulkerson algorithm

Keep augmenting flow along an augmenting path until there is no augmenting path.
Represent the flow attribute using the usual dot-notation, but on an edge: $(u, v).f$.

FORD-FULKERSON(G, s, t)

for all $(u, v) \in G.E$

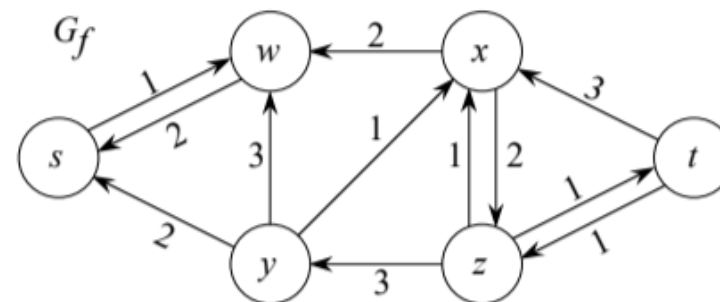
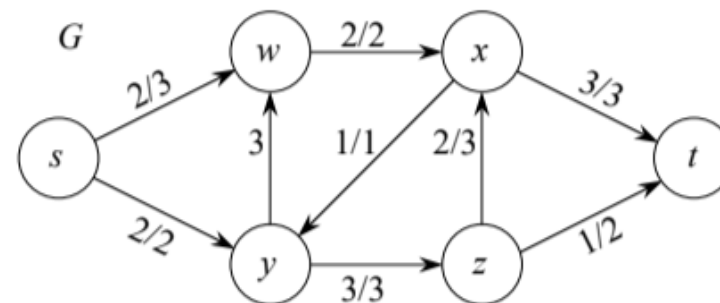
$(u, v).f = 0$

while there is an augmenting path p in G_f
 augment f by $c_f(p)$

Analysis

If capacities are all integer, then each augmenting path raises $|f|$ by ≥ 1 . If max flow is f^* , then need $\leq |f^*|$ iterations $\Rightarrow$ time is $O(E |f^*|)$.

Note that this running time is *not* polynomial in input size. It depends on $|f^*|$, which is not a function of $|V|$ and $|E|$.



الگوریتم ادموند کارپ

Edmonds-Karp algorithm

Do FORD-FULKERSON, but compute augmenting paths by BFS of G_f . Augmenting paths are shortest paths $s \rightsquigarrow t$ in G_f , with all edge weights = 1.

Edmonds-Karp runs in $O(VE^2)$ time.

To prove, need to look at distances to vertices in G_f .

Let $\delta_f(u, v)$ = shortest path distance u to v in G_f , with unit edge weights.

Lemma

For all $v \in V - \{s, t\}$, $\delta_f(s, v)$ increases monotonically with each flow augmentation.

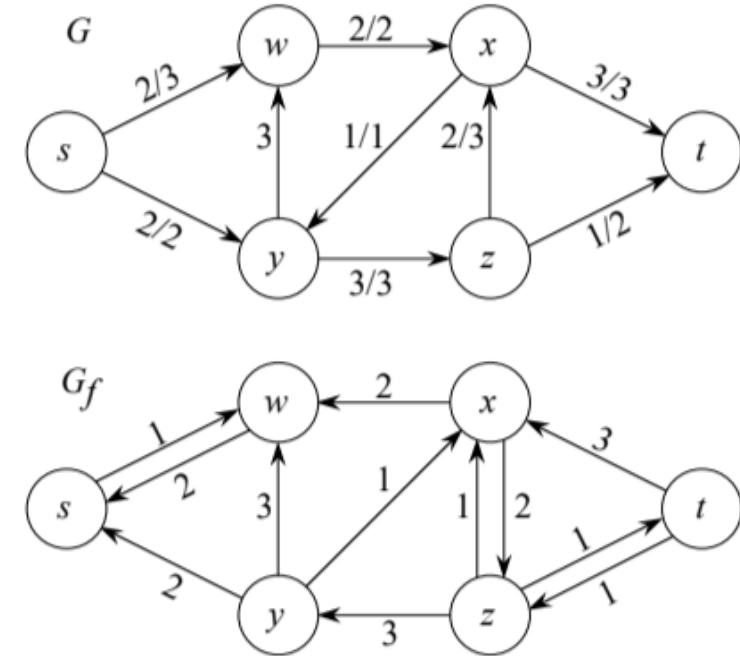
Proof Suppose there exists $v \in V - \{s, t\}$ such that some flow augmentation causes $\delta_f(s, v)$ to decrease. Will derive a contradiction.

Let f be the flow before the first augmentation that causes a shortest-path distance to decrease, f' be the flow afterward.

Let v be a vertex with minimum $\delta_{f'}(s, v)$ whose distance was decreased by the augmentation, so $\delta_{f'}(s, v) < \delta_f(s, v)$.

Let a shortest path s to v in $G_{f'}$ be $s \rightsquigarrow u \rightarrow v$, so $(u, v) \in E_{f'}$ and $\delta_{f'}(s, v) = \delta_{f'}(s, u) + 1$. (Or $\delta_{f'}(s, u) = \delta_{f'}(s, v) - 1$.)

Since $\delta_{f'}(s, u) < \delta_{f'}(s, v)$ and how we chose v , we have $\delta_{f'}(s, u) \geq \delta_f(s, u)$.



الگوریتم ادموند کارپ

Since $\delta_{f'}(s, u) < \delta_{f'}(s, v)$ and how we chose v , we have $\delta_{f'}(s, u) \geq \delta_f(s, u)$.

Claim

$(u, v) \notin E_f$.

Proof If $(u, v) \in E_f$, then

$$\begin{aligned} \delta_f(s, v) &\leq \delta_f(s, u) + 1 \quad (\text{triangle inequality}) \\ &\leq \delta_{f'}(s, u) + 1 \\ &= \delta_{f'}(s, v), \end{aligned}$$

contradicting $\delta_{f'}(s, v) < \delta_f(s, v)$.

■ (claim)

How can $(u, v) \notin E_f$ and $(u, v) \in E_{f'}$?

The augmentation must increase flow v to u .

Since Edmonds-Karp augments along shortest paths, the shortest path s to u in G_f has (v, u) as its last edge.

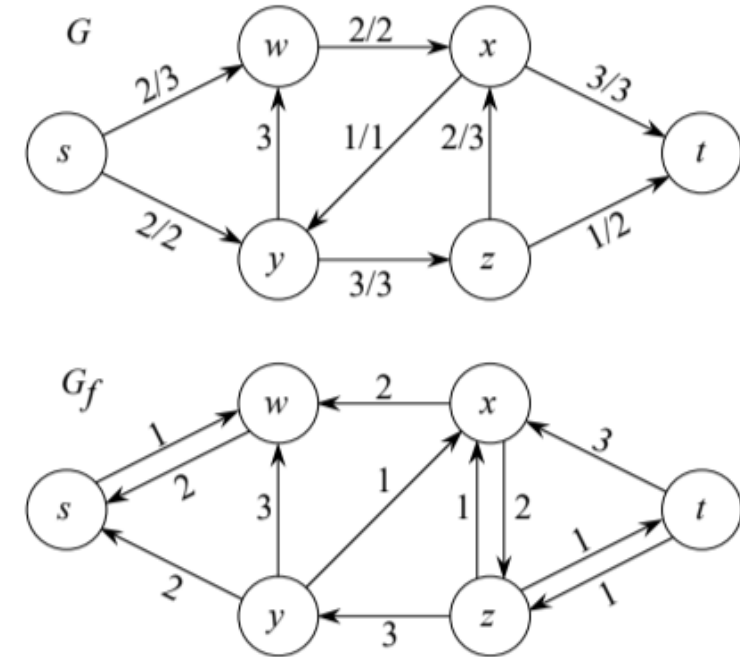
Therefore,

$$\begin{aligned} \delta_f(s, v) &= \delta_f(s, u) - 1 \\ &\leq \delta_{f'}(s, u) - 1 \\ &= \delta_{f'}(s, v) - 2, \end{aligned}$$

contradicting $\delta_{f'}(s, v) < \delta_f(s, v)$.

Therefore, v cannot exist.

■ (lemma)



الگوریتم ادموند کارپ

Theorem

Edmonds-Karp performs $O(VE)$ augmentations.

Proof Suppose p is an augmenting path and $c_f(u, v) = c_f(p)$. Then call (u, v) a **critical** edge in G_f , and it disappears from the residual network after augmenting along p .

≥ 1 edge on any augmenting path is critical.

Will show that each of the $|E|$ edges can become critical $\leq |V|/2$ times.

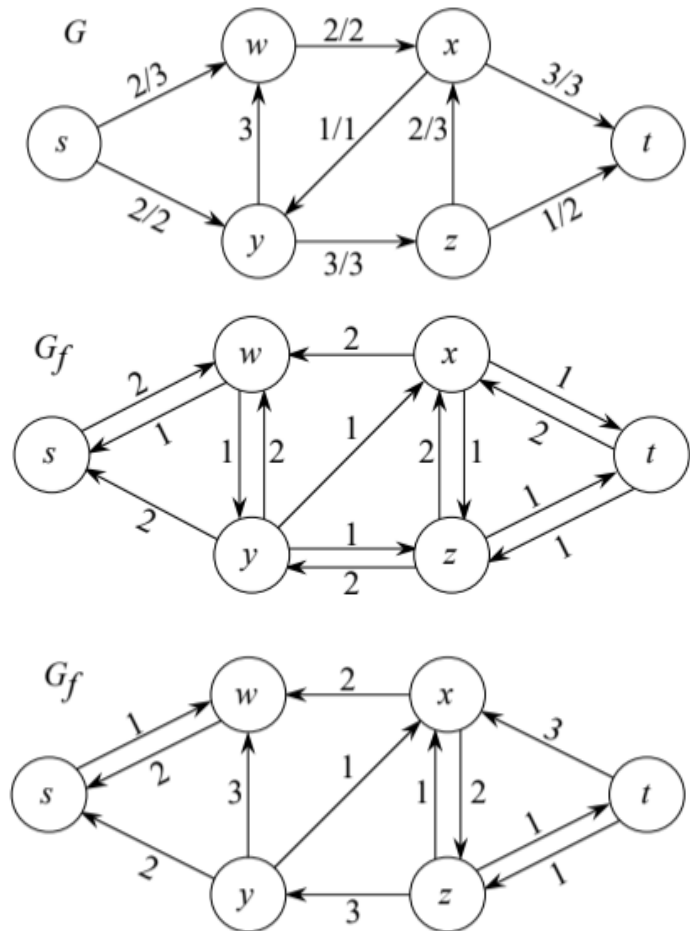
Consider $u, v \in V$ such that either $(u, v) \in E$ or $(v, u) \in E$ or both. Since augmenting paths are shortest paths, when (u, v) becomes critical first time, $\delta_f(s, v) = \delta_f(s, u) + 1$.

Augment flow, so that (u, v) disappears from the residual network. This edge cannot reappear in the residual network until flow from u to v decreases, which happens only if (v, u) is on an augmenting path in $G_{f'}$: $\delta_{f'}(s, u) = \delta_{f'}(s, v) + 1$. (f' is flow when this occurs.)

By lemma, $\delta_f(s, v) \leq \delta_{f'}(s, v) \Rightarrow$

$$\begin{aligned} \delta_{f'}(s, u) &= \delta_{f'}(s, v) + 1 \\ &\geq \delta_f(s, v) + 1 \\ &= \delta_f(s, u) + 2. \end{aligned}$$

Therefore, from the time (u, v) becomes critical to the next time, distance of u from s increases by ≥ 2 . Initially, distance to u is ≥ 0 , and augmenting path can't have s, u , and t as intermediate vertices.



الگوریتم ادموند کارپ

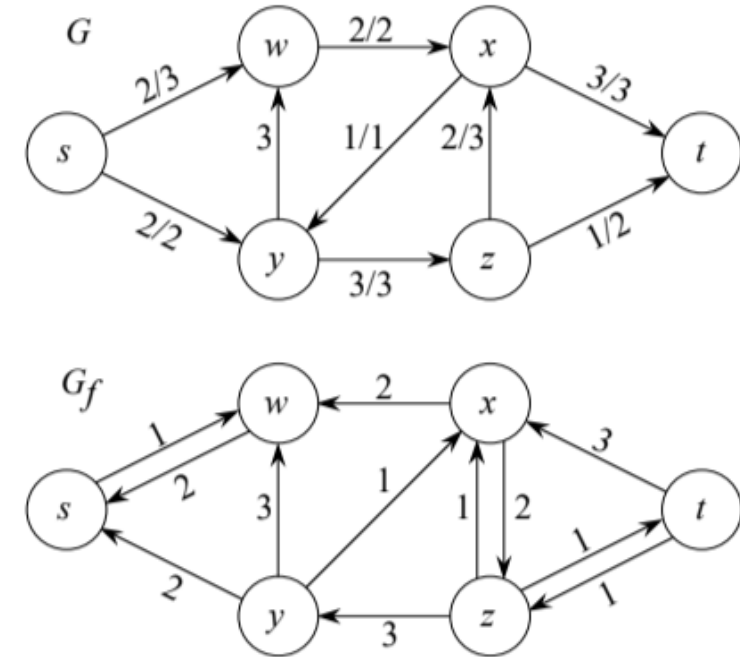
Therefore, from the time (u, v) becomes critical to the next time, distance of u from s increases by ≥ 2 . Initially, distance to u is ≥ 0 , and augmenting path can't have s, u , and t as intermediate vertices.

Therefore, until u becomes unreachable from source, its distance is $\leq |V| - 2 \Rightarrow$ after (u, v) becomes critical the first time, it can become critical $\leq (|V| - 2)/2 = |V|/2 - 1$ times more $\Rightarrow (u, v)$ can become critical $\leq |V|/2$ times.

Since $O(E)$ pairs of vertices can have an edge between them in residual network, total # of critical edges during execution of Edmonds-Karp is $O(VE)$. Since each augmenting path has ≥ 1 critical edge, have $O(VE)$ augmentations. ■ (theorem)

Use BFS to find each augmenting path in $O(E)$ time $\Rightarrow O(VE^2)$ time.

Can get better bounds.



نقطه ضعف فورد فولکرسون

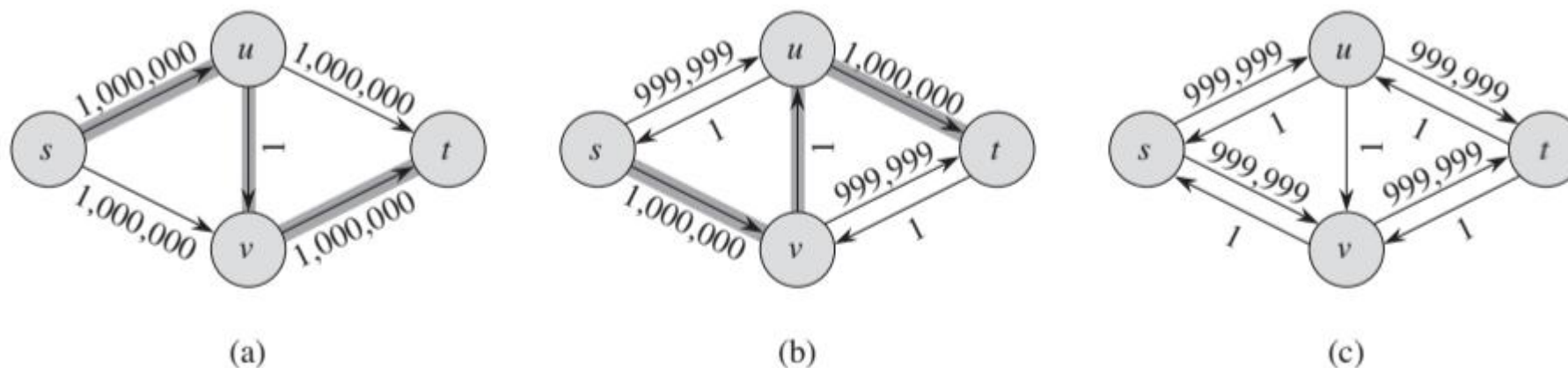
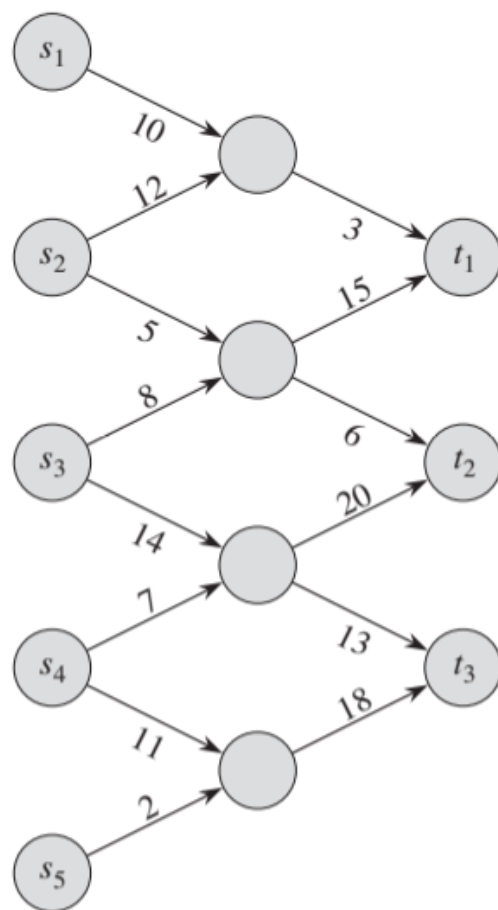
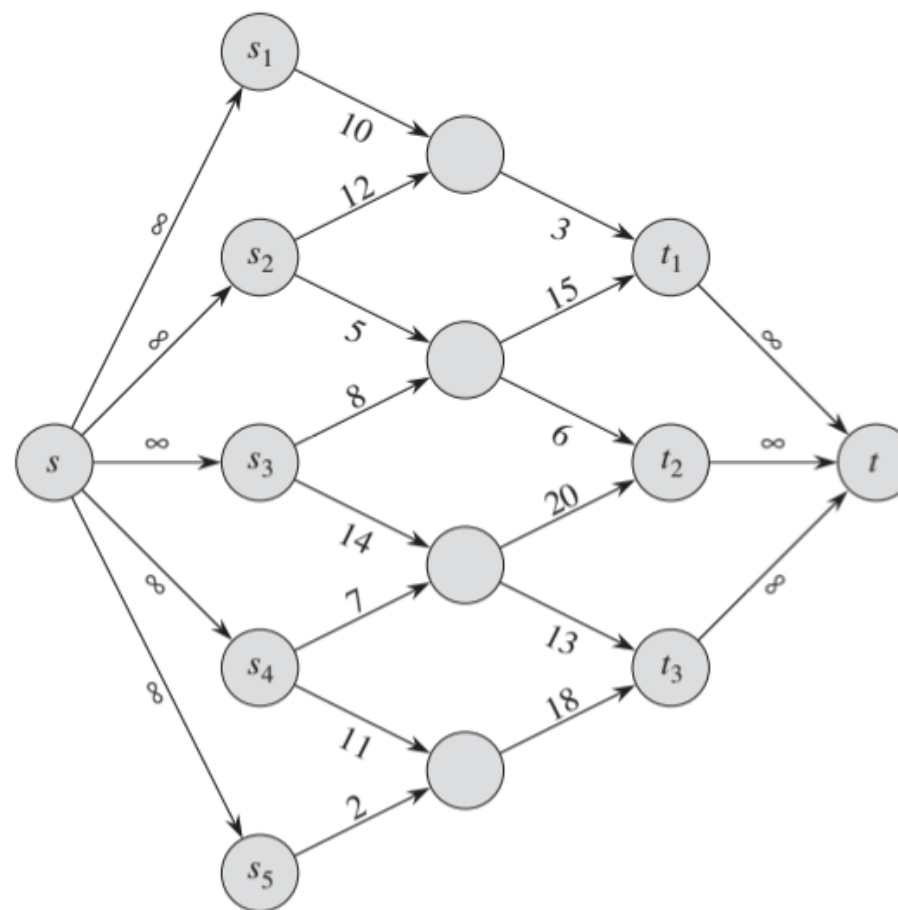


Figure 26.7 (a) A flow network for which FORD-FULKERSON can take $\Theta(E |f^*|)$ time, where f^* is a maximum flow, shown here with $|f^*| = 2,000,000$. The shaded path is an augmenting path with residual capacity 1. (b) The resulting residual network, with another augmenting path whose residual capacity is 1. (c) The resulting residual network.

جریان بیشینه با چند منبع و چند سینک



(a)

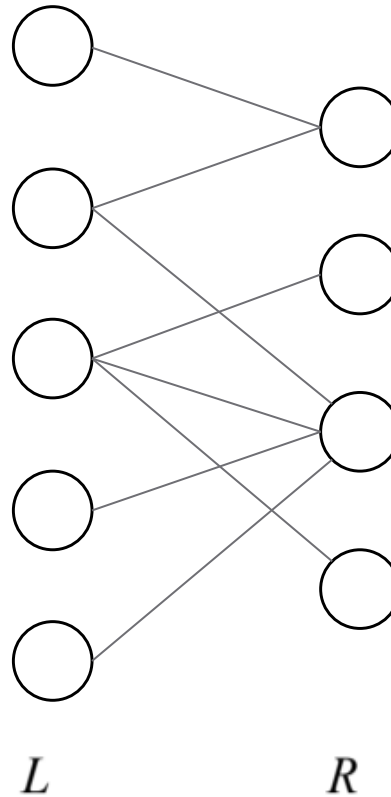


(b)

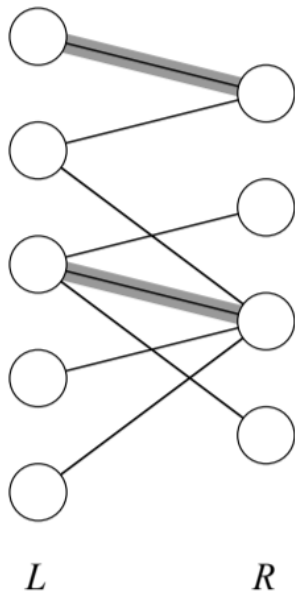
گراف دوبخشی Bipartite Graph

Example of a problem that can be solved by turning it into a flow problem.

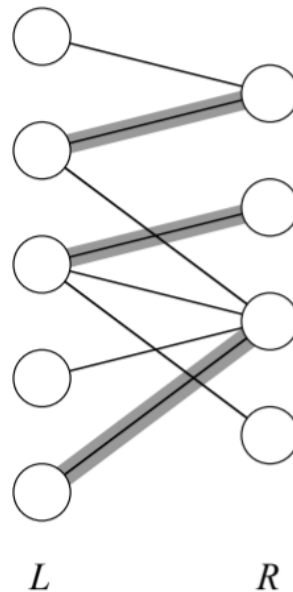
$G = (V, E)$ (undirected) is **bipartite** if we can partition $V = L \cup R$ such that all edges in E go between L and R .



Maximum Bipartite Matching



matching



maximum matching

A **matching** is a subset of edges $M \subseteq E$ such that for all $v \in V$, ≤ 1 edge of M is incident on v . (Vertex v is **matched** if an edge of M is incident on it; otherwise **unmatched**).

Maximum matching: a matching of maximum cardinality. (M is a maximum matching if $|M| \geq |M'|$ for all matchings M' .)

Maximum Bipartite Matching

Problem

Given a bipartite graph (with the partition), find a maximum matching.

Application

Matching planes to routes.

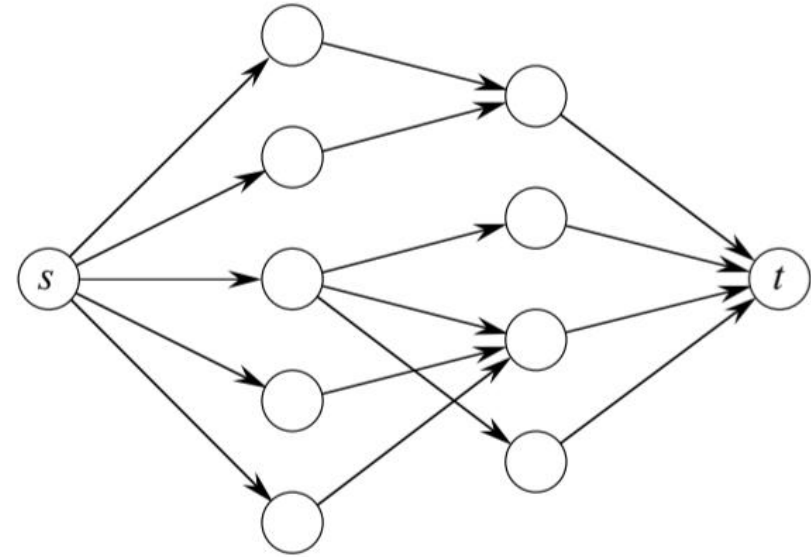
- L = set of planes.
- R = set of routes.
- $(u, v) \in E$ if plane u can fly route v .
- Want maximum # of routes to be served by planes.

Solution: Maximum Bipartite Matching

We define the *corresponding flow network* $G' = (V', E')$

Given G , define flow network $G' = (V', E')$.

- $V' = V \cup \{s, t\}$.
- $E' = \{(s, u) : u \in L\} \cup \{(u, v) : (u, v) \in E\} \cup \{(v, t) : v \in R\}$.
- $c(u, v) = 1$ for all $(u, v) \in E'$.



Each vertex in V has ≥ 1 incident edge $\Rightarrow |E| \geq |V|/2$.

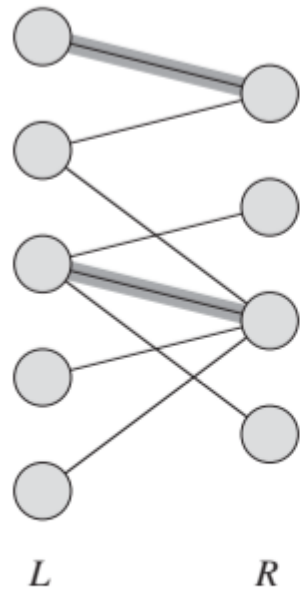
Therefore, $|E| \leq |E'| = |E| + |V| \leq 3|E|$.

Therefore, $|E'| = \Theta(E)$.

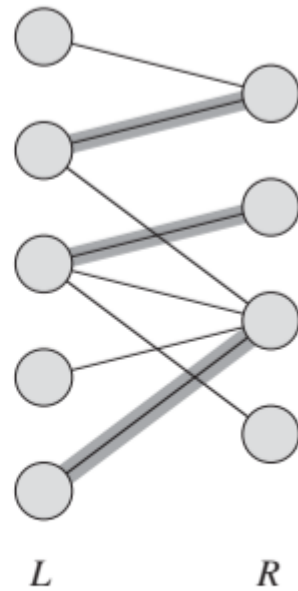
Find a max flow in G' .

Use edges that carry flow of 1 in matching.

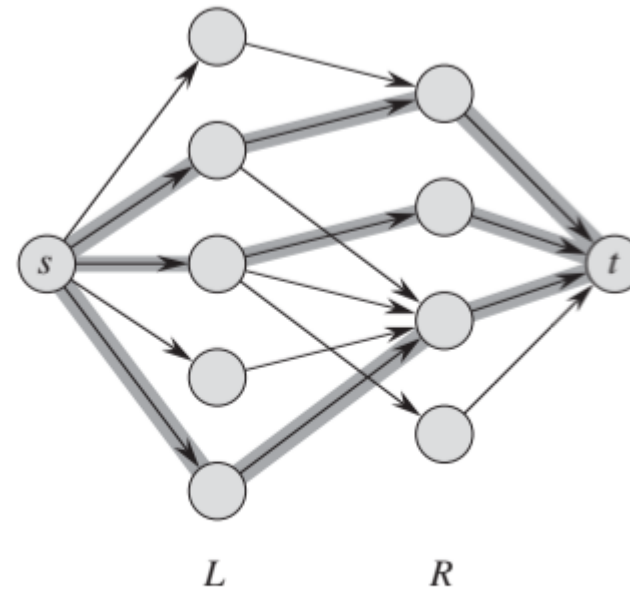
Solution: Maximum Bipartite Matching



(a)



(b)



(c)